Design Manual

Smart Vehicle Tracking and Management System



Group 23:

E/20/272 Niroshan B.G.L.

E/20/456 Dulaj U.P.S.D.

Table of content

1. Introduction
    1.1 Project Overview
    1.2 Objective and Scope
    1.3 Solution Architecture
    1.4 Data Flow
    1.5 Security

2. Software Infrastructure
    2.1 Frontend
        2.1.2 Key Features
        2.1.3 Technology Stack
        2.1.4 UI Design
    2.2 Backend
        2.2.1 Introduction
        2.2.2 Key Features
        2.2.3 Technology Stack
        2.2.4 Modular Architecture and Data Flow
        2.2.5 RESTful API Endpoints

3. Cloud Deployment Architecture and Procedure
    3.1 Architecture Overview
        3.1.1 Multi-Tier Architecture Design
        3.1.2 Cloud Infrastructure Components
    3.2 Azure Server Setup
        3.2.1 Azure Account Configuration
        3.2.2 App Service Creation
        3.2.3 Static Web App for Frontend
    3.3 Server Environment Configuration
        3.3.1 Environment Variables Setup
        3.3.2 SSL/TLS Configuration
        3.3.3 CORS Configuration
    3.4 Database, Frontend and Backend Deployment
        3.4.1 MongoDB Atlas Setup
        3.4.2 Firebase Realtime Database Configuration
        3.4.3 Backend Deployment Process
        3.4.4 Frontend Deployment Process
    3.5 Service Management

3.5.1 Application Monitoring
3.5.2 Logging Configuration
3.5.3 Auto-scaling Configuration
3.5.4 Backup and Recovery
3.6 Client Configuration
3.6.1 Web Application Configuration
3.6.2 Mobile Application Configuration
3.6.3 IoT Device Configuration
3.6.4 Security Configuration
3.6.5 Performance Optimization

4. Hardware infrastructure
4.1 Central Control unit
4.1.1 Introduction
4.1.2 Hardware Components
4.1.3 Firmware
4.2 3D-Designs
4.3 PCB Designs
4.3.1 Sensors
4.3.1.1 GPS Module
4.3.1.2 ADXL 345 Sensor
4.3.1.3 DHT11 Temperature and Humidity Sensor
4.4 Actuators
4.4.1 SIM800l GSM Module
4.4.2 Buzzer Alarm

5. Accident and Tamper Detection System Using Sensor Data
5.1 Introduction
5.2 Detection Logic

6. Testing
6.1 Software Testing
6.2 Hardware Testing

7. Source Code Repositories

1 Introduction

1.1 Project Overview

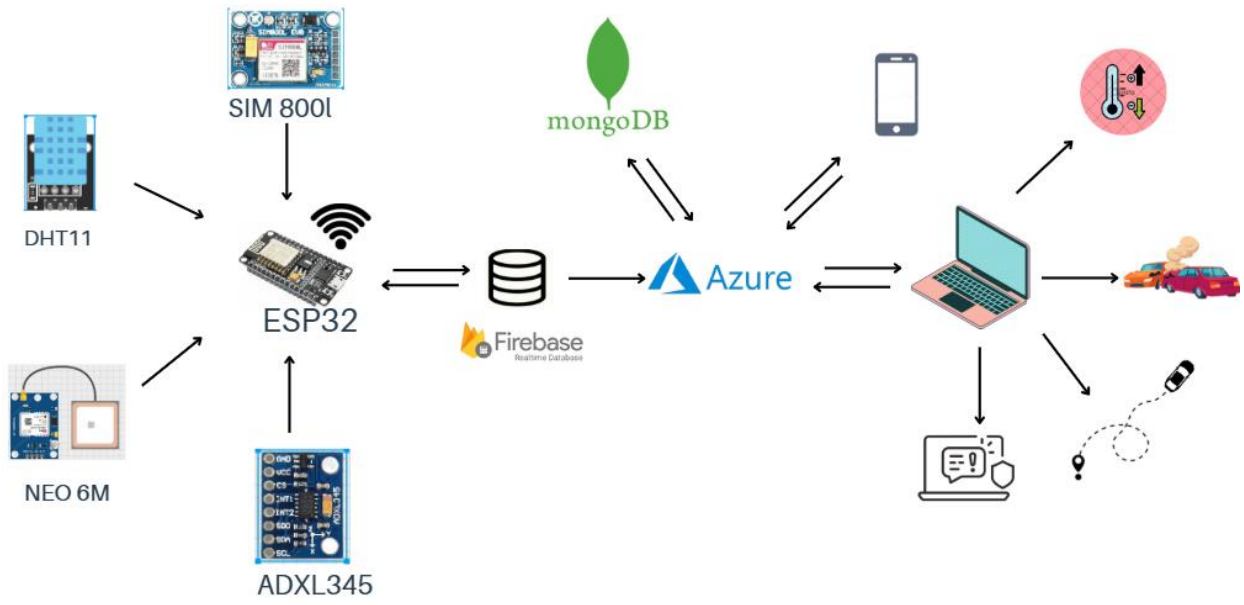
The Smart Vehicle Tracking and Management System is an advanced IoT based solution designed to enhance vehicle safety, monitoring, and operational efficiency. This system integrates real-time GPS tracking, accident detection, tamper alerts, environmental sensing (temperature and humidity), and remote data monitoring via cloud services. Utilizing an ESP32 microcontroller, the system communicates with various sensor modules.A driver side mobile application enables real time interaction, while a web based dashboard provides location tracking, alerts, and vehicle status monitoring. All hardware components are integrated into a custom PCB enclosed in a 3D-printed case for secure in vehicle deployment.

1.2 Objective and Scope

The objective of this design manual is to guide users, developers, and stakeholders in implementing, deploying, and utilizing the Smart Vehicle Tracking and Management System. It covers all key components, including system architecture, sensor integration, data flow, PCB and enclosure design, cloud connectivity, and mobile/web interfaces. The scope includes configuration, deployment, and effective use of the system for reliable, real-time vehicle tracking, safety monitoring, and management.

1.3 Solution Architecture

The Smart Vehicle Tracking and Management System is designed as a modular, distributed IoT solution that combines hardware, firmware, and cloud platforms to enable real-time vehicle monitoring and safety management. The system architecture is based on the ESP32 microcontroller, which interfaces with sensors such as the NEO-6M GPS module, ADXL345 accelerometer, and DHT11 temperature and humidity sensor. Data is transmitted via the SIM800L GSM module and stored on cloud platforms such as Firebase, , Azure, and MongoDB for data storage,processing, and enabling real-time tracking, alerting, and data visualization through both web dashboards and a dedicated driver side mobile application.



1. Device Layer

User Devices:

- Driver Mobile Application (Android): Displays real time vehicle location, tasks, environmental status, and emergency functions.
- Owner Web Dashboard (React-based): Allows real time tracking, system monitoring,alerts(accident/tamper/speed.etc), and historical alert viewing.
- Communication with the backend is established via REST APIs and Firebase Realtime Database for live updates.

2. API & Backend Layer

API & Data Services:

- Built using Node.js with Express.js, the backend exposes RESTful APIs for both mobile and web clients.
- Responsible for handling data from the ESP32 (via GSM), processing incoming alerts (accident/tamper), and updating cloud storage.

Database:

- Firebase Realtime Database is used to store dynamic data, including live GPS coordinates, sensor readings, event alerts, and real-time system status.
- MongoDB is used to store static data, such as driver profiles, vehicle information, and system configuration settings.

3. Cloud & Messaging Layer

Firebase Cloud Platform:

- Acts as the core cloud backend for real-time database storage and communication.
- Synchronizes data across devices and enables alert/event notification systems.

SMS Alerts and Internet via GSM:

The SIM800L GSM module, connected to the ESP32, serves dual purposes:

- It sends SMS alerts for critical events such as accidents, tampering, or unauthorized disconnection, ensuring reliable notification even when cloud or Wi-Fi access is unavailable.
- It also provides GPRS-based internet connectivity, enabling the ESP32 to transmit real-time data (GPS location, sensor values, and system status) to the cloud database (e.g., Firebase) when Wi-Fi is not present.

4. Edge Device Layer

ESP32 Microcontroller (Core Processing Unit):

- Interfaces with all sensors and modules (GPS, ADXL345, DHT11, SIM800L).
- Executes onboard logic to detect sudden acceleration (accidents), vibrations (tamper), and collects environmental data.
- Sends data periodically to Firebase and critical alerts via GSM SMS.

5. Sensor & Module Layer

Sensors and Modules Connected to ESP32:

- NEO-6M GPS Module – Provides real time location coordinates for vehicle tracking.
- ADXL345 Accelerometer – Detects sharp impacts (accidents) and abnormal vibrations (tampering or unauthorized access).
- DHT11 Sensor – Monitors temperature and humidity levels inside the vehicle.
- SIM800L GSM Module – Enables SMS based alert notifications and GPRS internet connectivity for cloud communication.
- TP4056 Charging Module – Manages safe charging of the onboard battery and provides regulated power to the ESP32 and connected modules.

- 3.7V Li-ion Battery – Serves as the primary or backup power source, ensuring uninterrupted operation during power loss.
- Buzzer – Emits audible alarms during critical events like temperature alerts or tampering.

6. Data Flow & Communication Protocols

Sensor Data Acquisition:

- ESP32 collects data from all sensors at set intervals and processes logic for accident/tamper detection.
- Regular data is sent to Firebase via GSM network (using HTTPS).
- Critical alerts (accidents/tampering) trigger both **Firebase updates** and **SMS messages** via SIM800L.

User Interaction:

- The driver mobile app retrieves real-time updates supports:
    - Viewing assigned tasks from the admin or owner.
    - Update the status of the tasks.

- The owner web dashboard provides:
    - Live vehicle tracking and environment monitoring
    - Full alert history, including all real-time incidents (accidents, tamper, etc)
    - Task assignment and system status controls

7. Cloud Integration

- **Azure Hosting:**
  The backend service (Node.js/Express) is deployed on Azure App Service, while the admin web dashboard is hosted using Azure Static Web Apps, ensuring high availability, scalability, and fast content delivery.
- **Mobile App Deployment:**
  The driver mobile application is built and deployed using Expo Application Services (EAS) for seamless cross-platform delivery and OTA (over-the-air) updates.

1.4 Data Flow

1. Sensors collect real-time vehicle data, including GPS coordinates, acceleration (for accident/tamper detection), and environmental conditions (temperature, humidity).
2. The ESP32 processes this data and determines if thresholds are exceeded (e.g., detect accident,detect tamper).
3. Regular sensor data is sent to Firebase via GPRS over the SIM800L module, while critical alerts (accident, tamper) trigger SMS notifications and immediate cloud updates.
4. The driver mobile app receives task assignments in real time.
5. The owner web dashboard displays live vehicle location, system status,system alerts and historical event logs.
6. All data is securely stored in the cloud using Firebase for dynamic data and MongoDB for static records, allowing for remote monitoring and long-term analysis.

1.5 Security

1. Authentication and Authorization with Role-Based Permissions

- User Authentication:

  All users must log in with valid credentials to access the system. Authentication ensures that only verified individuals can view or interact with vehicle data.
- Role-Based Authorization:

  Access is restricted based on user roles:
  - Admins can manage vehicle records, assign tasks, and monitor all drivers and alerts.
  - Drivers can only access their own tasks, status updates, and system alerts. This prevents unauthorized access to administrative functions and sensitive data.

2. Unique User Identification

- Personalized Access:

  Each user (admin or driver) is assigned a unique user ID upon registration. This ID links to all associated tasks, vehicle data, and alert history.
- Secure Data Isolation:

  The system ensures that users can only view and manage data relevant to their own role and assigned vehicles, maintaining data privacy and integrity.

The system is designed to handle multiple vehicles and users, making it suitable for both small and medium businesses.

2. Software Infrastructure

2.1 Frontend

2.1.1 Key Features

- Real-Time Vehicle Tracking: Continuous live tracking of vehicles using GPS data with location updates shown on an interactive map.
- Event Detection and Alerts: Accident and tamper detection via ADXL345, with real-time alerts sent through both SMS and Firebase to the owner dashboards.
- Task Management: Admins can assign delivery or service tasks to drivers. Drivers receive tasks and status updates via the mobile app.
- Environment Monitoring and Speed Tracking:DHT11 sensor monitors temperature and humidity inside the vehicle. GPS-based speed monitoring allows the owner to track real-time vehicle speed for safety and compliance.
- Alert Notifications: Instant notifications sent to owners when abnormal events (accident, tampering, disconnect) are detected.
- Historical Data Visualization: The system stores and displays past location history, alert logs, and sensor data for admin review.

2.1.2 Technology Stack

Component	**Technology/Package**	**Purpose/Role**
Language	JavaScript / TypeScript	Main language for frontend (React, Expo)
Web Framework	React.js	Component-based architecture for modular and scalable UI

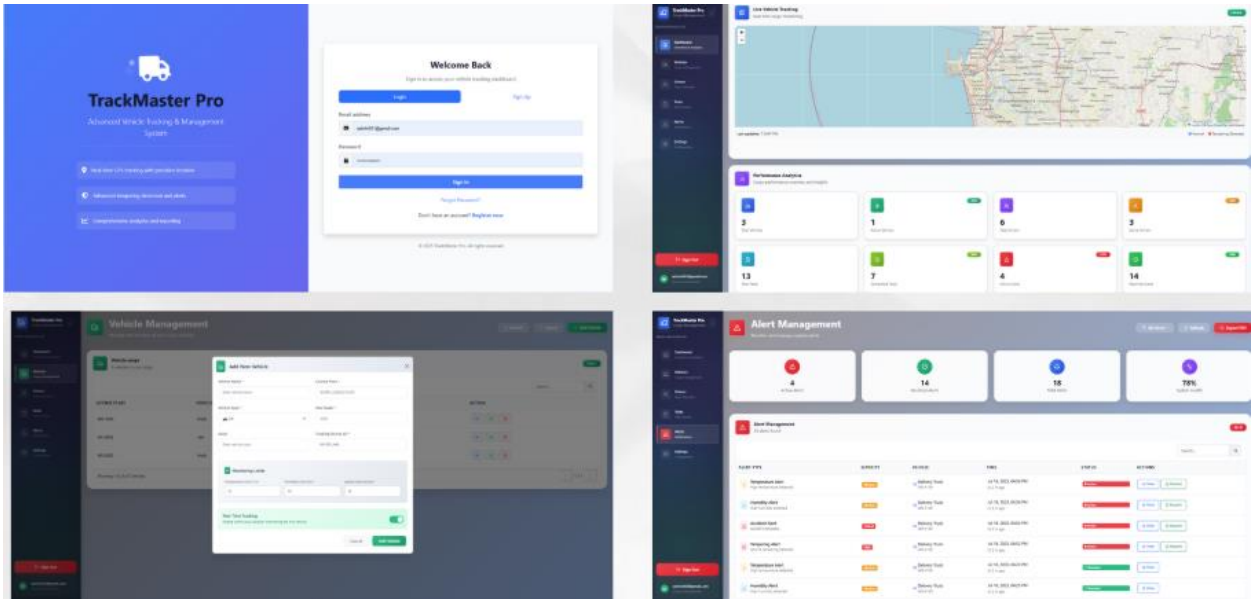
Mobile Framework	Expo (React Native)	Cross-platform mobile app for drivers
Map Library	Leaflet.js	Real-time vehicle location tracking with markers and popups

2.1.3 UI Design

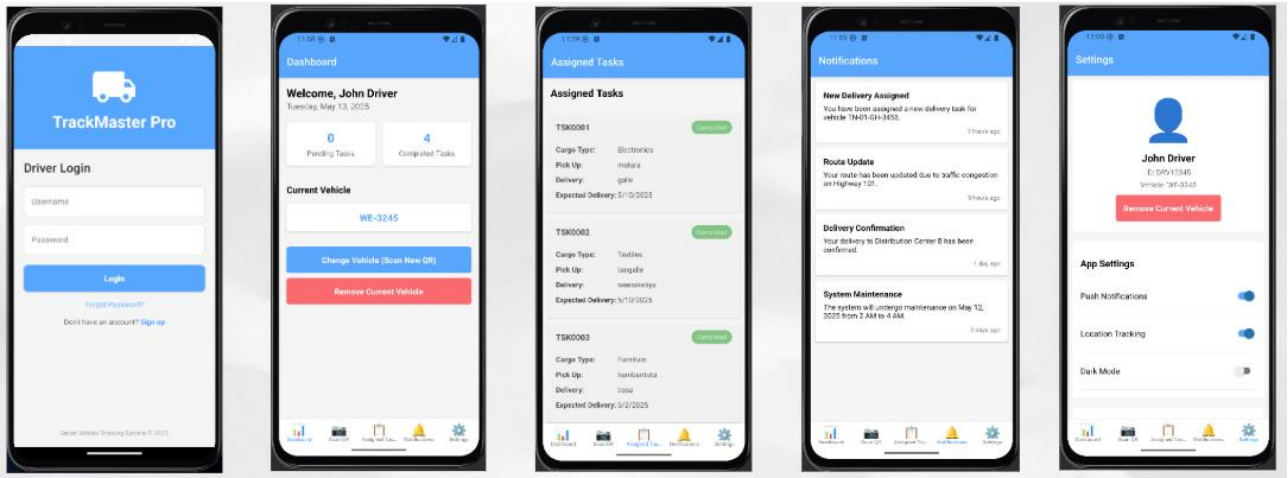
Figma Design:

The system's interface is designed in Figma for both web and mobile platforms, prioritizing usability for both owners and drivers.

- Owner/Admin Web App: Includes vehicle tracking screen, task manager, alert log viewer, and driver management panel.
- Driver Mobile App: Screens include task list and vehicle status dashboard.
- The interface ensures seamless navigation, clear map-based tracking, and alert visibility on all screen sizes.



Admin Web App Design



Driver mobile App Design

2.2 Backend

2.2.1 Introduction

The backend of the Vehicle Tracking and Management System serves as the core engine that connects the frontend applications (web and mobile), embedded IoT devices, and cloud databases. It handles API communication, data validation, user authentication, and real time data synchronization. This backend ensures seamless coordination between vehicle-mounted sensors, the cloud, and user dashboards.

2.2.2 Key Features

1. RESTful API Services:Exposes secure and well-structured RESTful APIs to allow smooth interaction between frontend apps and backend logic.

2. Real-Time Data Handling:Processes real-time location, speed, environmental, and accelerometer data from vehicles and updates the Firebase Realtime Database instantly.

3. User Authentication and Role-Based Authorization: Validates user identities and manages permissions based on roles (Admin, Driver), ensuring access control to features like tracking, task assignment, and alert viewing.

4. Alert Generation and Logging: Backend evaluates incoming sensor data (ADXL345, DHT11, GPS) for anomalies (accidents, tampering, overheating) and triggers alerts accordingly.

5. Hybrid Data Storage Strategy:

    - MongoDB: Stores static and structured data such as user profiles, vehicle metadata, historical alerts, and task records.

- Firebase Realtime Database: Manages dynamic and real-time data like GPS coordinates, speed updates, environmental sensor readings, and system status flags.

6. Device Command Relay:Routes commands from the frontend (like task assignment) to devices via APIs or push notifications.

2.2.3 Technology Stack

Component	**Technology**	**Purpose/Role**
Backend Runtime	Node.js	Backend environment for running JavaScript code
API Framework	Express.js	RESTful routing, request/response handling
Database (Static)	MongoDB	Stores user, vehicle, task, and alert data
Realtime Database	Firebase Realtime DB	Stores and syncs real-time GPS and sensor data
Cloud Hosting	Azure	Scalable backend deployment

The backend is primarily implemented in JavaScript (Node.js/Express), with Mongo DB for data storage.

2.2.4 Modular Architecture and Data Flow

- Route Layer:Handles HTTP requests from web/mobile apps and routes them to the appropriate controller.

- Controller Layer:Authenticates users, validates incoming requests, and triggers the business logic layer.
- Service Layer:Implements the system logic including device monitoring, alert generation, task management, and permission checks.
- Database Layer:Communicates with MongoDB and Firebase to retrieve/update records based on the type of data (static vs dynamic).
- Data Flow Overview:
    - Vehicle sensors (GPS, ADXL345, DHT11) send data to the ESP32.
    - ESP32 pushes dynamic data (e.g., GPS, temperature, motion) to Firebase Realtime Database.
    - Backend reads this data, performs logic checks, and updates MongoDB with alerts or task logs.
    - Frontend fetches updated data and visuals via REST APIs and Firebase listeners.

2.2.5 RESTful API Endpoints

The backend provides a comprehensive set of RESTful API endpoints organized by functionality. All endpoints follow standard HTTP methods and return JSON responses with appropriate status codes.

Authentication & User Management

Method	Endpoint	Description
POST	`/api/auth/register`	Register new admin user
POST	`/api/auth/login`	User login authentication
GET	`/api/auth/profile`	Get current user profile
PUT	`/api/auth/profile`	Update user profile

Vehicle Management

Method	Endpoint	Description
POST	`/api/vehicles`	Register new vehicle
GET	`/api/vehicles`	Get all vehicles with status
GET	`/api/vehicles/:id`	Get specific vehicle details
GET	`/api/vehicles/:id/location`	Get real-time vehicle location
GET	`/api/vehicles/:id/history`	Get vehicle trip history

Task Management

Method	Endpoint	Description
POST	`/api/tasks`	Create new task
GET	`/api/tasks`	Get all tasks with filters
PUT	`/api/tasks/:id`	Update task information
POST	`/api/tasks/:id/start`	Start task execution
POST	`/api/tasks/:id/complete`	Complete task

Alert & Notification Management

Method	Endpoint	Description
GET	`/api/alerts`	Get all alerts with filters
POST	`/api/alerts`	Create manual alert
PUT	`/api/alerts/:id/acknowledge`	Acknowledge alert
PUT	`/api/alerts/:id/resolve`	Resolve alert

Reports & Analytics

Method	Endpoint	Description
GET	`/api/reports/fleet-summary`	Get fleet overview report
GET	`/api/reports/driver-performance`	Get driver performance report
POST	`/api/reports/generate`	Generate custom report
GET	`/api/reports/:id/download`	Download generated report

Real-time & System

Method	Endpoint	Description
GET	`/api/realtime/vehicles`	Get all vehicles real-time status
WebSocket	`/socket.io`	Real-time bidirectional communication
GET	`/api/config/health`	System health check

- Authentication: All protected endpoints require JWT token in Authorization header.
- Rate Limiting: 60 requests/minute for general APIs, 5 requests/minute for auth endpoints.
- Response Format: Standard JSON with success/error status and appropriate HTTP codes (200, 400, 401, 404, 500).

3. Cloud Deployment Architecture and Procedure

3.1 Architecture Overview

3.1.1 Multi-Tier Architecture Design

The Smart Vehicle Tracking and Management System follows a modern three-tier cloud architecture deployed on Microsoft Azure:

- Presentation Tier: React.js web application and React Native mobile app
- Application Tier: Node.js/Express.js backend services with RESTful APIs
- Data Tier: MongoDB Atlas for persistent data and Firebase Realtime Database for IoT sensor data

3.1.2 Cloud Infrastructure Components

1. Azure App Service: Hosts the Node.js backend application
2. Azure Static Web Apps: Hosts the React.js admin frontend
3. Azure CDN: Content delivery network for static assets
4. MongoDB Atlas: Cloud-hosted NoSQL database for user data, vehicles, tasks, and alerts
5. Firebase Realtime Database: Real-time IoT sensor data streaming
6. Azure Application Insights: Monitoring and analytics

3.2 Azure Server Setup

3.2.1 Azure Account Configuration

1. Subscription: Azure for Students or Pay-As-You-Go
2. Resource Group: TrackMasterPro
3. Region: South India (for optimal latency)
4. Pricing Tier: B1 Basic for development, P1V2 for production

3.2.2 App Service Creation

```
# Azure CLI commands for setup

az group create --name TrackMasterPro --location southindia

az appservice plan create --name vehicle-tracking-plan --resource-group
vehicle-tracking-rg --sku B1

az webapp create --name trackmasterpro --resource-group vehicle-tracking-
rg --plan vehicle-tracking-plan --runtime "NODE|18-lts"
```

3.2.3 Static Web App for Frontend

```
az staticwebapp create \
  --name vehicle-tracking-frontend \
  --resource-group vehicle-tracking-rg \
  --source https://github.com/your-repo/admin-frontend \
  --location southindia \
  --branch main \
  --app-location "/code/admin-frontend" \
  --build-location "dist"
```

3.3 Server Environment Configuration

3.3.1 Environment Variables Setup

Backend environment configuration in Azure App Service.

```
# Application Settings in Azure Portal
NODE_ENV=production
PORT=80
MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/vehicletracking
JWT_SECRET=your-secure-jwt-secret
FIREBASE_API_KEY=your-firebase-api-key
FIREBASE_DATABASE_URL=https://yp-group-23-default-rtdb.asia-southeast1.firebasedatabase.app
CORS_ORIGIN=https://your-frontend-domain.azurestaticapps.net
```

3.3.2 SSL/TLS Configuration

- HTTPS Enforcement: Enabled by default on Azure App Service
- Custom Domain: Configure if needed with SSL certificate
- Security Headers: Implemented in Express.js middleware

3.3.3 CORS Configuration

```
// Backend CORS setup for production
app.use(cors({
  origin: [
    "https://vehicle-tracking-frontend.azurestaticapps.net",
    "https://trackmasterpro-faethrezd6cvauee.southindia-01.azurewebsites.net"
  ],
  methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization"],
  credentials: true
}));
```

3.4 Database, Frontend and Backend Deployment

3.4.1 MongoDB Atlas Setup

1. Cluster Configuration: M0 Sandbox (Free tier) or M10 for production
2. Region: South Asia (Mumbai) for optimal performance
3. Network Access: Whitelist Azure App Service IP ranges
4. Database User: Create dedicated user with read/write permissions
5. Connection String:

   mongodb+srv://username:password@cluster.mongodb.net/vehicletracking

3.4.2 Firebase Realtime Database Configuration

```
// Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyBf-ZdT6dz-39syOFnvWVzByU5B1Dhn9ws",
  authDomain: "yp-group-23.firebaseapp.com",
  databaseURL: "https://yp-group-23-default-rtdb.asia-southeast1.firebasedatabase.app",
  projectId: "yp-group-23",
  storageBucket: "yp-group-23.firebasestorage.app",
  messagingSenderId: "422453667678",
  appId: "1:422453667678:web:d5c5fdcc005fb2354591cf"
};
```

3.4.3 Backend Deployment Process

```yaml
# GitHub Actions workflow for backend deployment
name: Deploy Backend to Azure
on:
  push:
    branches: [main]
    paths: ['code/vehicle-tracking-backend/**']

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Setup Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '18'
      - name: Install dependencies
        run: |
          cd code/vehicle-tracking-backend
          npm install
      - name: Deploy to Azure
        uses: azure/webapps-deploy@v2
        with:
          app-name: 'trackmasterpro'
          publish-profile: ${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}
          package: 'code/vehicle-tracking-backend'
```

3.4.4 Frontend Deployment Process

```yaml
# GitHub Actions workflow for frontend deployment
name: Deploy Frontend to Azure Static Web Apps
on:
  push:
    branches: [main]
    paths: ['code/admin-frontend/**']

jobs:
  build_and_deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Build And Deploy
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${{ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN }}
          repo_token: ${{ secrets.GITHUB_TOKEN }}
          action: "upload"
          app_location: "/code/admin-frontend"
          api_location: ""
          output_location: "dist"
```

3.5 Service Management

3.5.1 Application Monitoring

- Azure Application Insights: Integrated for performance monitoring
- Custom Metrics: API response times, database query performance
- Error Tracking: Automatic exception logging and alerting
- Availability Tests: Health check endpoints monitoring

3.5.2 Logging Configuration

```
// Winston logger configuration for production
const winston = require('winston');
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'app.log' })
  ]
});
```

3.5.3 Auto-scaling Configuration

- Scale-out Rules: CPU > 70% for 5 minutes
- Scale-in Rules: CPU < 30% for 10 minutes
- Instance Limits: Minimum 1, Maximum 3 instances
- Cool-down Period: 5 minutes between scaling operations

3.5.4 Backup and Recovery

- MongoDB Atlas: Automatic daily backups with 7-day retention
- Firebase: Automatic backup through Firebase console
- Application Code: Version controlled in GitHub with release tags
- Configuration: Environment variables backed up securely

3.6 Client Configuration

3.6.1 Web Application Configuration

```
// Production API configuration
const API_CONFIG = {
  baseURL: 'https://trackmasterpro-faethrezd6cvauee.southindia-01.azurewebsites.net/',
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json'
  }
};
```

3.6.2 Mobile Application Configuration

```
// React Native API host configuration
const getBaseURL = () => {
  if (Platform.OS === 'android') {
    return 'https://trackmasterpro-faethrezd6cvauee.southindia-01.azurewebsites.net/';
  } else if (Platform.OS === 'ios') {
    return 'https://trackmasterpro-faethrezd6cvauee.southindia-01.azurewebsites.net/';
  } else {
    return 'https://trackmasterpro-faethrezd6cvauee.southindia-01.azurewebsites.net/';
  }
};
```

3.6.3 IoT Device Configuration

```
// ESP32 configuration for cloud connectivity
const char* serverURL = "https://trackmasterpro-faethrezd6cvauee.southindia-01.azurewebsites
const char* wifiSSID = "YourWiFiNetwork";
const char* wifiPassword = "YourWiFiPassword";

// GSM configuration for backup connectivity
const char* apn = "your-gsm-apn";
const char* gprsUser = "";
const char* gprsPass = "";
```

3.6.4 Security Configuration

- API Authentication: JWT tokens with 24-hour expiration
- HTTPS Enforcement: All client communications encrypted
- CORS Policy: Restrictive origin allowlist
- Rate Limiting: 100 requests per minute per IP
- Input Validation: Server-side validation for all API endpoints

3.6.5 Performance Optimization

- CDN Integration: Static assets served through Azure CDN
- Caching Strategy: Redis cache for frequently accessed data
- Database Indexing: Optimized queries with proper indexing
- Connection Pooling: Database connection optimization
- Gzip Compression: Enabled for all text-based responses

4. Hardware infrastructure

4.1 Central Control unit

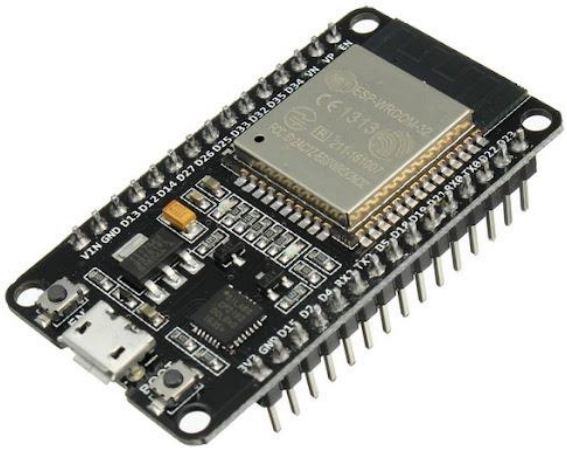
4.1.1 Introduction

The Central Control Unit (CCU) of the Vehicle Tracking and Management System is built around the ESP32 microcontroller, which serves as the core processing and communication hub within the vehicle. It is responsible for collecting data from various onboard sensors, including GPS module,DHT11 sensor ,ADXL 345 accelerometer In addition to data acquisition, the CCU also handles control operations.The use of the ESP32 provides key benefits such as,Low power consumption,Built-in Wi-Fi and Bluetooth for future scalability,Sufficient processing power to handle sensor fusion and real-time decision making.

4.1.2 Hardware Components

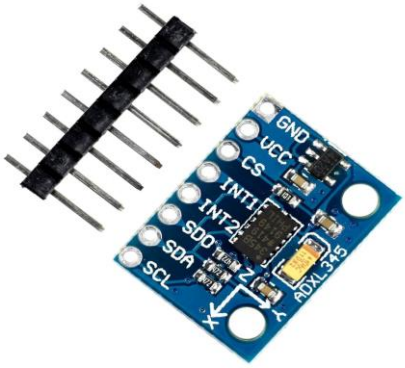
- **ESP32 Dev1 Board**

Acts as the Central Control Unit (CCU) of the system. Offers built-in Wi-Fi and Bluetooth, dual core processing, multiple GPIOs, and ADC/PWM support, making it ideal for real-time sensor data processing and wireless communication with cloud/mobile interfaces.



• ADXL345 Accelerometer

A 3-axis digital accelerometer used for accident detection and tamper/motion sensing. Communicates via I²C/SPI and offers high resolution output with adjustable sensitivity and interrupts.



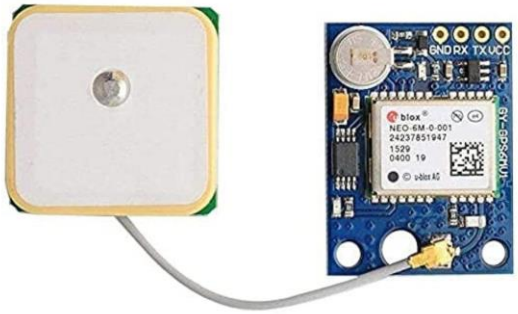
• SIM800L GSM Module (EVB Version)

Provides cellular connectivity to the ESP32 for internet access via GPRS. It operates on 2G networks and communicates through AT commands over UART. This module enables the ESP32 to push sensor data to cloud platforms (e.g., Firebase), ensuring real-time vehicle tracking and remote monitoring.



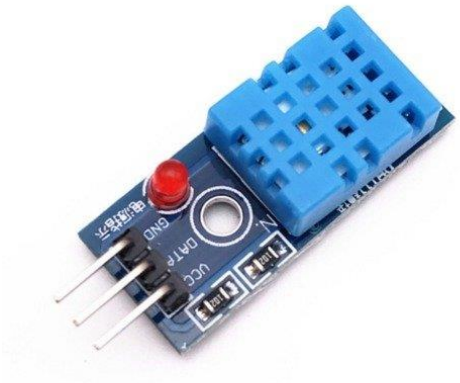
• NEO-6M GPS Module

A GPS receiver used to obtain real-time location and speed data of the vehicle. Outputs NMEA sentences via UART and provides high positioning accuracy with quick acquisition time.



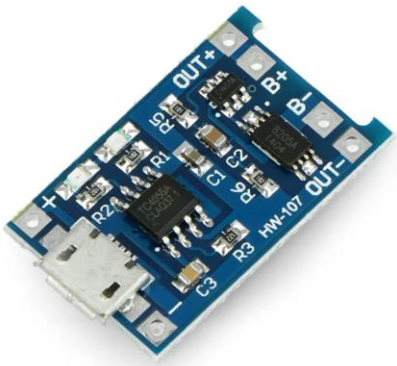
• DHT11 Temperature and Humidity Sensor

A low cost digital sensor used to monitor ambient temperature and humidity inside the vehicle. It communicates via a single-wire digital interface with the ESP32. This sensor is essential for tracking internal cargo conditions, helping to prevent damage or spoilage of sensitive goods due to environmental fluctuations.



• TP4056 Lithium Battery Charging Module

A 1-cell Li-ion/LiPo battery charger module with micro-USB input. Ensures safe charging of a 3.7V lithium battery with overcharge, overdischarge, and short-circuit protection.



• 3.7V Li-ion Battery

Rechargeable power source for the system. Provides backup power to ensure uninterrupted tracking and alert functions even when vehicle power is lost.



• I²C LCD Display (16x2)

Used to show live sensor values, GPS coordinates, system status, or alert messages. Communicates with the ESP32 using I²C protocol, reducing GPIO usage.



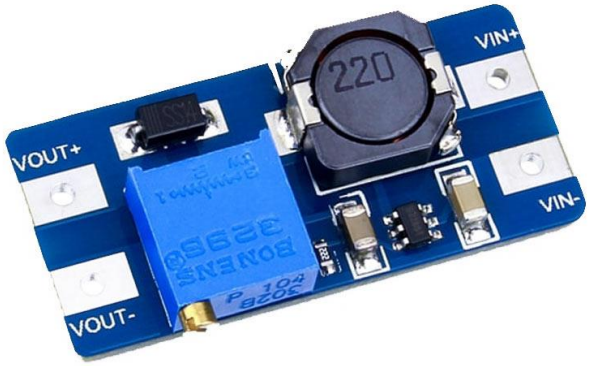
• Passive Buzzer

Acts as a local audible alert system. It is triggered during critical events such as unauthorized movement (tampering) or temperature-related warnings, effectively alerting nearby personnel to immediate issues within or around the vehicle.



• Boost Converter (DC-DC Step-Up Module)

Steps up the 3.7V battery voltage to 5V to power 5V components like the LCD or SIM800L module. Ensures voltage stability for critical components.



4.1.3 Firmware

The firmware for the ESP32 microcontroller is developed using the Arduino framework (C/C++), enabling seamless integration with multiple sensors and modules for real-time vehicle monitoring. The firmware handles data acquisition, processing, wireless communication, and alert generation.

System Initialization

- Initializes UART interfaces for SIM800L (GSM), GPS module, and ADXL345 accelerometer (I2C/SPI).
- Configures GPIOs for sensor inputs and buzzer output.
- Loads stored configuration values such as alert thresholds and Firebase endpoints.
- Initializes watchdog timers and sensor readiness checks to ensure stable startup.

Sensor Data Acquisition

- Reads data periodically from:
    - GPS Module (via TinyGPS++): Latitude, longitude, speed, date/time.
    - ADXL345 Accelerometer: Measures X, Y, Z axis acceleration for motion, impact, and tamper detection.
    - DHT11 Sensor: Monitors temperature and humidity inside the vehicle or cargo compartment.
- Monitors power stability and reconnection events using internal status flags.

Data Processing

- Filters and smooths accelerometer data to detect sudden impacts (accidents) and motion patterns (tampering).
- Converts GPS raw data into usable coordinates and speed.
- Applies temperature/humidity thresholds to detect unsafe cargo conditions.
- Formats sensor data into structured JSON strings for cloud or alert communication.

Communication (GPRS via SIM800L)

- Establishes GPRS connectivity using AT commands.
- Pushes sensor data periodically to **Firebase Realtime Database** using HTTPS REST API.
- Communicates with the cloud using secure GET/POST requests, retrying automatically upon failure.

Safety and Error Handling

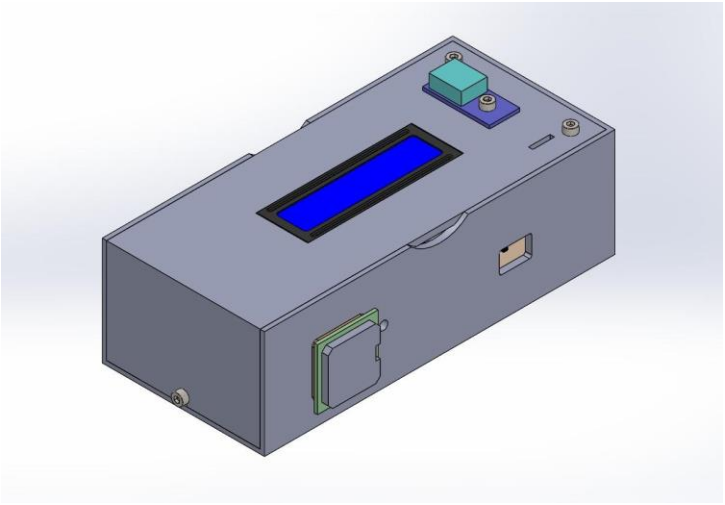
- Continuously monitors SIM800L, GPS, and sensor communication status.
- Triggers alert routines (buzzer + SMS + Firebase update) on fault detection.
- Logs error events for debugging and maintenance.

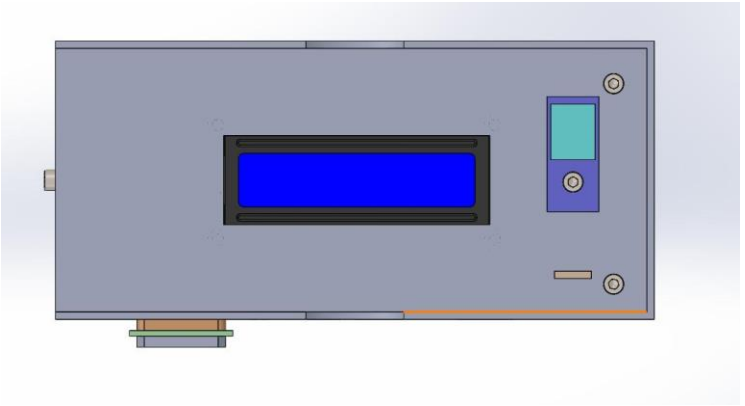
Scheduled Tasks

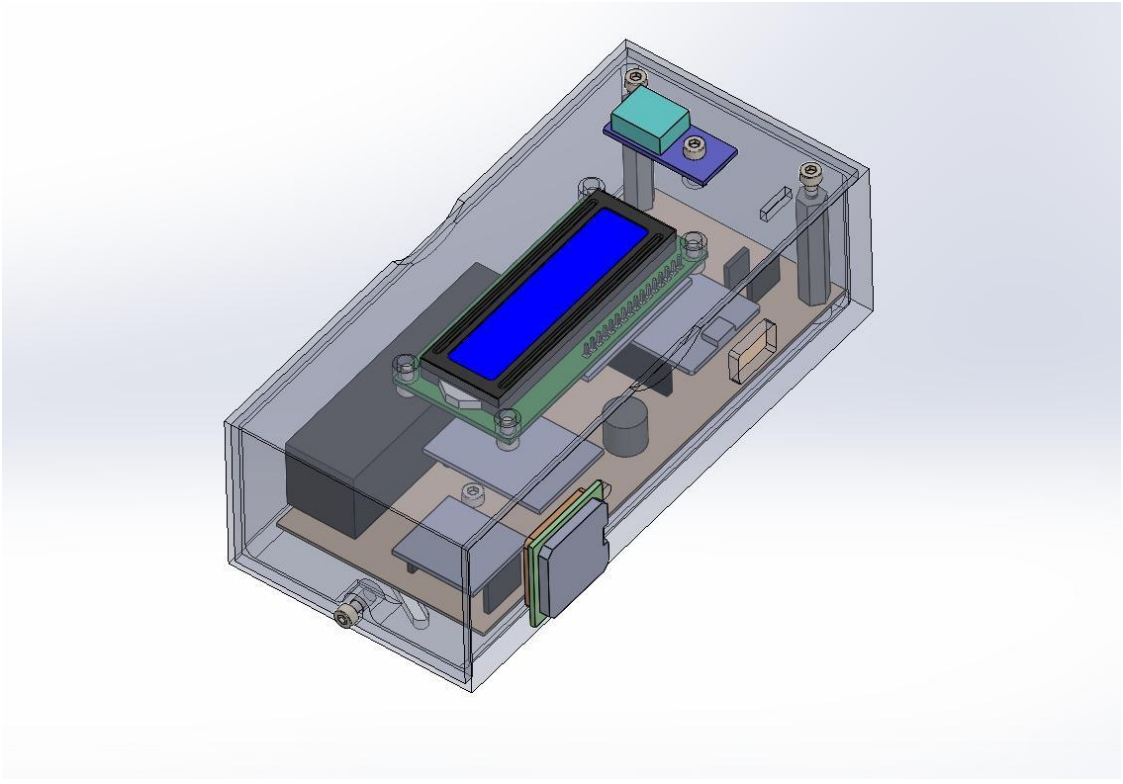
- Executes sensor data collection every 5 seconds.
- Pushes location and environmental data to Firebase at regular intervals.
- Checks thresholds and alert conditions continuously within the main loop.

The ESP32 firmware runs in a real-time loop, optimized for low power consumption and robust network handling.

4.2 3D-Designs

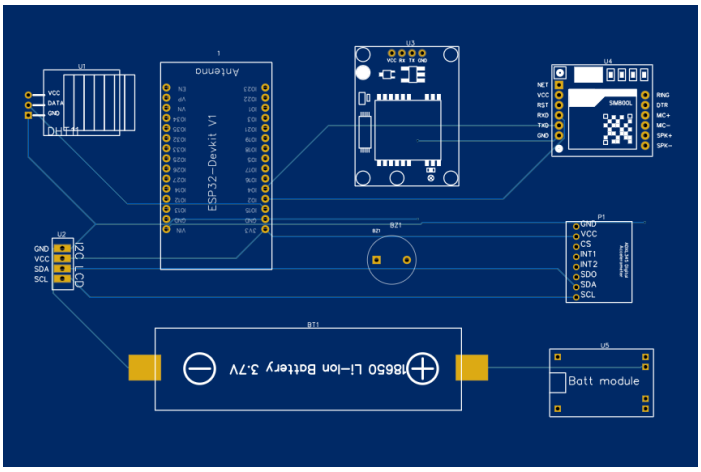




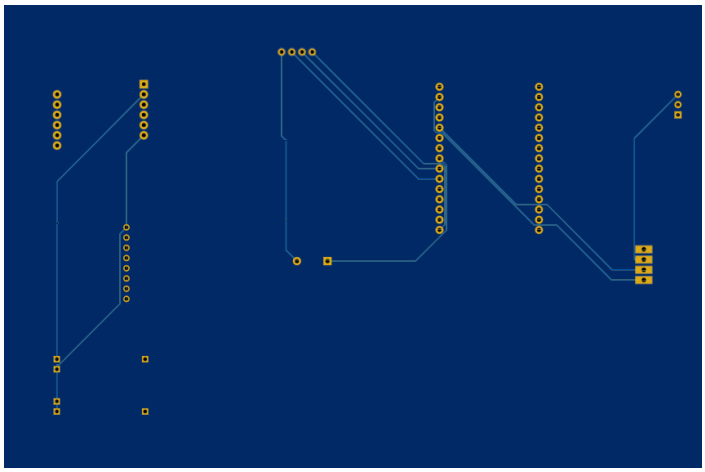


Covers the mechanical design and housing for the Vehicle Tracking Unit. It includes CAD models and a 3D-printed enclosure designed to securely mount all internal components. The sensors-connected PCB board is mounted inside the enclosure, ensuring protection, durability, shock resistance, and ease of maintenance, while maintaining a compact form suitable for vehicle installation.

4.3 PCB Designs



Top view



Bottom view

The custom designed PCB, integrates all sensor modules and components onto a compact single-layer board. It features thickened power traces for current handling, strategically placed decoupling and bypass capacitors for noise reduction, and clearly labeled connectors for easy assembly. This PCB layout ensures electrical reliability and supports the scalability of the system for mass production.

4.3.1 Sensors

4.3.1.1 GPS Module

- The GPS tracking system uses a standard NEO-6M GPS module that continuously receives satellite data and transmits location information (latitude, longitude, speed) to the ESP32 via UART communication.
- The ESP32 reads the GPS data using the TinyGPS++ library, processes it, and packages it into JSON format. This location data is periodically pushed to Firebase using the HTTPS REST API over GPRS via the SIM800L GSM module.
- The real-time location is displayed on the web dashboard developed using React, where vehicle movement is shown on a map using Google Maps API integration.

4.3.1.2 ADXL345 Accelerometer (Accident and Tamper Detection)

- The ADXL345 accelerometer is connected to the ESP32 via the I2C interface and is used to monitor vehicle motion and orientation. It detects sudden changes in acceleration (potential accidents) and unusual device movements (potential tampering or unauthorized movement).
- The ESP32 runs a custom logic to filter out noise and debounce the signals. If a threshold is exceeded (e.g., sudden G-force), an accident or tamper alert is triggered.
- The alert is sent to the Firebase real-time database, and optionally, an emergency SMS is sent via the SIM800L module to the registered contact number. A buzzer is also activated to provide local feedback.

4.3.1.3 DHT11 Temperature and Humidity Sensor

- The DHT11 sensor is connected to the ESP32 via a digital GPIO pin and periodically measures the in-vehicle ambient temperature and humidity.

- The ESP32 processes the sensor readings and uploads the data to Firebase using HTTPS REST API. This environmental data is displayed on the owner's web dashboard for monitoring vehicle cabin conditions and can help in detecting overheating or harsh environments that may affect sensitive cargo or electronics.

4.4 Actuators

4.4.1 SIM800L GSM Module (for SMS Alerts)

- The SIM800L GSM module is used for SMS-based alerting and communication in the system. It is connected to the ESP32 via UART.
- When an accident or tamper event is detected, the ESP32 sends an AT command sequence to the SIM800L to dispatch a predefined alert notification to the owner or emergency contact.
- The module operates on 2G networks and is powered through a filtered and regulated power rail to prevent brownouts during transmission. It enables vehicle owners to receive critical alerts even without internet access.

4.3.2 Buzzer Alarm

- A small piezo buzzer is connected to a digital GPIO pin on the ESP32. It is used as a local audio alert mechanism.
- When a tamper event is detected via the ADXL345, the ESP32 activates the buzzer to emit an immediate audible alarm. This helps deter unauthorized handling or alerts nearby personnel.
- The buzzer control is handled through a simple ON/OFF pulse pattern using digitalWrite logic on the ESP32.

5. Accident and Tamper Detection System Using Sensor Data

5.1 Introduction

The system uses an ADXL345 accelerometer integrated with an ESP32 microcontroller to monitor sudden impacts (accidents) and unauthorized tampering (e.g., shocks, orientation changes, free fall). Sensor data is processed locally on the ESP32, which triggers alerts and logs events with GPS location and timestamp information. Alerts are sent in real-time to Firebase, enabling remote monitoring, while a buzzer and LCD provide local feedback.

5.2 Detection Logic

- **Accident Detection:**
  - Uses the ADXL345's tap interrupt to detect sudden, strong shocks (impacts).
  - An interrupt service routine (ISR) triggers when a tap (impact) is detected.
  - On detection, event data including GPS coordinates and timestamp are sent to Firebase.
  - The LCD shows an "ACCIDENT ALERT!" message.
- **Tamper Detection:**
  - Continuously monitors accelerometer data for:
    - **Free fall:** When total acceleration drops below a threshold (approx. 2 m/s^2), indicating possible device removal or fall.
    - **Sudden shocks:** Large changes in acceleration beyond a defined shock threshold.
    - **Orientation changes:** Significant deviations from baseline acceleration representing rotation or movement.

- Tamper events are debounced (minimum delay between detections) to prevent false alarms.
- Detected tampering triggers Firebase data updates and buzzer activation.
- The LCD shows "TAMPER ALERT!" when tampering is detected.

- **Baseline Calibration:**
  - At startup, the system records the initial stable acceleration values to use as a baseline for detecting orientation changes and shocks.

6. Testing

6.1 Software testing

6.1.1 Used Softwares

- **6.1.1.1 Postman**

  Used to test REST API endpoints between the backend and web/mobile app

  Verified correctness of URLs, HTTP methods (GET, POST), and headers

  Tested request and response payloads for GPS location, sensor alerts, and user login/registration

6.1.2 Functionality Testing

- Ensured all connected modules (GPS, DHT11, ADXL345, SIM800L) transmit data accurately to the backend or Firebase
- Verified real-time accident and tamper detection alerts via SMS (GSM module), buzzer, LCD, and web app
- Validated live GPS tracking updates and proper coordinate mapping

6.1.3 User Interface Testing (Web App - Owner Side)

- Tested login, registration, password handling, and session management
- Verified correct rendering of real-time GPS coordinates on Google Maps
- Checked sensor data visualization (temp/humidity/alert state)
- Validated alert notification flow and acknowledgment buttons
- Tested responsive design on various screen sizes (desktop, tablet, mobile)

6.1.4 Usability Testing

- Observed users (vehicle owners) interacting with the web application to evaluate:
  - Ease of navigation and clarity of map view
  - Intuitiveness of alert acknowledgment
  - Effectiveness of real-time updates and alert popups
- Collected feedback to improve user experience (e.g., larger map, clear sensor labels, SMS alert confirmation status)

6.2 Hardware testing

6.2.1 GPS Module Testing

- Location Accuracy Test: Compare GPS coordinates from the module with known locations (Google Maps reference).
- Signal Stability: Evaluate GPS fix time and signal retention under different environments (open sky, urban, indoor).
- Location Logging: Verify that the system correctly records and transmits GPS data to Firebase in real time.

6.2.2 Sensor & Actuator Testing

- Accelerometer (ADXL345): Test accident and tamper detection logic (tap, free fall, orientation change) for reliability and repeatability.
- Speed Monitoring: Simulate varying speeds and confirm alerts are triggered when thresholds are exceeded.
- Buzzer & LCD Display: Ensure buzzer activates and messages display correctly for different alerts (tamper, over temperature).

6.2.3 System Performance Testing

- ESP32 Processing Speed: Measure event processing time, data logging latency, and response time for interrupts.
- Real-Time Data Handling: Verify that ESP32 can handle simultaneous sensor inputs and send data to Firebase without lag.

6.2.4 Connectivity Testing

- GSM (SIM800L) Module: Test SMS and Firebase communication reliability in different network conditions.
- Firebase Integration: Confirm real-time updates of all sensor values, event logs, and alert statuses.

6.2.5 Power Supply Testing

- Voltage Stability: Test system operation under full load using battery and external supply.
- Overload & Heat: Monitor for overheating or voltage drops during continuous operation or high current draw.

6.2.6 Field Environment Testing

- Vehicle Mount Test: Mount the device in a vehicle and evaluate vibration resistance, tamper alert accuracy, and GPS logging during actual motion.
- Outdoor Testing: Expose the system to varying temperatures, movements, and GPS conditions to test reliability in real-world use.

7. Source Code Repositories

Github repository url

https://github.com/cepdnaclk/e20-3yp-Smart-Vehicle-Tracking-and-Management-System

Github pages url

https://cepdnaclk.github.io/e20-3yp-Smart-Vehicle-Tracking-and-Management-System/